Aerial Guardian

Person Detection and Tracking from Drone Footage

Technical Report — Assignment Submission

Research Engineer (Computer Vision and Deep Learning)

Author	Abhay Kumar Yadav
Roll No	22104066
Date	May 2026
Repository	github.com/Abhay2003abhay/aerial-guardian

1. Introduction

Aerial surveillance from unmanned aerial vehicles (UAVs) presents unique challenges for computer vision systems. Detecting and tracking persons from a moving drone platform is significantly harder than standard ground-level detection due to three primary challenges: small target size, camera ego-motion, and real-time performance constraints.

The VisDrone 2019 Multi-Object Tracking (MOT) dataset captures these challenges realistically, with persons often appearing at just 10-30 pixels in height across 1344x756 resolution frames. Standard object detectors trained on COCO or similar datasets perform poorly on such small targets.

This report describes the Aerial Guardian pipeline, which combines YOLOv8n for detection, SAHI tiled inference for small object improvement, and ByteTrack for robust multi-object tracking with persistent IDs across frames.

2. Dataset and Analysis

2.1 VisDrone 2019 MOT Dataset

The VisDrone 2019 MOT validation set contains 7 drone footage sequences captured from UAVs at varying altitudes over urban scenes. The dataset includes pedestrians, cyclists, and vehicles in both sparse and crowded environments.

Property	Value
Total sequences	7
Total frames	2,846
Image resolution	1344 x 756 pixels
Frame rate	30 FPS
Annotation format	frame_id, target_id, x, y, w, h, score, category
Person categories	Pedestrian (1), People (2)
Total annotations	23,877

2.2 Key Analysis Finding

Data exploration revealed that the average person bounding box is only 33 x 60 pixels on a 1344x756 frame. The minimum height observed was 14 pixels, with most persons appearing between 10-40 pixels tall. This confirmed that standard object detectors would miss most targets, justifying the SAHI approach.

Metric	Width (px)	Height (px)

Average	32.7	60.3
Minimum	11	14
Maximum	101	144

3. System Architecture

3.1 YOLOv8n — Detection Model

YOLOv8 nano (YOLOv8n) was selected as the detection backbone. At 6.2MB it is well within the 300MB model size constraint and provides fast inference on CPU. The nano variant uses a lightweight CSPDarknet backbone with depthwise separable convolutions, enabling real-time performance on resource-constrained hardware.

Key properties:

- Model size: 6.2 MB (limit: 300 MB)
- Architecture: CSPDarknet backbone with PANet neck
- Input resolution: 640x640 (standard)
- Trained on: COCO dataset (80 classes including person)
- Only class 0 (person) used for inference

3.2 SAHI — Slicing Aided Hyper Inference

The core challenge was that persons occupied only 10-30 pixels in height. Standard YOLOv8n, trained on images where persons are typically much larger, missed most targets. SAHI addresses this by dividing each frame into overlapping tiles, running detection on each tile independently, then merging predictions using NMS.

SAHI configuration used:

- Slice size: 320x320 pixels
- Overlap ratio: 0.2 (20%) in both dimensions
- Tiles per frame: 15 tiles on 1344x756 input
- Result merging: NMS with IoU threshold 0.5

3.3 ByteTrack — Multi-Object Tracker

ByteTrack was selected for multi-object tracking. Unlike DeepSORT, it does not require a re-identification network, making it significantly lighter and faster — critical for edge deployment. ByteTrack uses a Kalman filter to predict each tracked person's position in the next frame based on motion history.

A key innovation in ByteTrack is that it uses both high and low confidence detections. Low-confidence detections that match existing tracks are used to recover persons that brief occlusion causes standard trackers to lose, significantly reducing ID switches.

3.4 Trajectory Visualization

Each tracked person is assigned a consistent color based on their track ID. A fading trajectory tail of up to 40 points is drawn behind each person, with older points rendered at lower opacity. This visualizes movement patterns and helps verify tracking continuity across frames.

4. Experiments and Results

4.1 Detection Comparison: Standard vs SAHI

Method	Persons Detected	FPS (CPU)	Notes
YOLOv8n standard	20	4.78	Misses most small persons
YOLOv8n + SAHI	39	0.24	+95% improvement

4.2 Full Tracking Pipeline Results

Metric	Value
Model	YOLOv8n
Model size	6.2 MB
Tracker	ByteTrack
Test sequence	uav0000086_00000_v (464 frames)
Average FPS (CPU)	4.83
Hardware	MacBook Air (CPU only)
Total unique IDs tracked	48
Confidence threshold	0.25
Trajectory tail length	40 frames
SAHI slice size	320x320

5. Discussion

5.1 Handling Small Object Detection

The primary challenge in this dataset is that persons appear at extremely small sizes — often just 10-30 pixels tall on a 1344x756 resolution frame. Standard YOLOv8n, despite being a strong detector, was trained on COCO where persons typically occupy a much larger portion of the image. Running it at 640x640 on full drone frames means small persons are compressed further, making detection unreliable.

SAHI solves this by running detection on 320x320 tiles with 20% overlap. Each tile contains a smaller region of the original frame, so persons that were 20px tall in the full frame appear proportionally larger in their tile. This nearly doubled detections from 20 to 39 persons per frame (+95% improvement).

5.2 Handling ID Switching

Drone ego-motion causes the background to shift every frame, making simple position-based tracking unreliable. ByteTrack addresses this using a Kalman filter that models each person's position and velocity. Even when detection fails for a few frames due to occlusion or motion blur, the Kalman filter continues to predict the person's location, maintaining their track ID.

ByteTrack's key innovation of using low-confidence detections for existing tracks further reduces ID switches. When a person is partially occluded, the detector may output a low-confidence prediction that would normally be discarded — ByteTrack uses these to keep existing tracks alive rather than losing them.

5.3 Edge Deployment on Jetson

For deployment on Jetson Nano or Jetson Xavier NX, the YOLOv8n model would first be exported to TensorRT format using model.export(format='engine', device=0). TensorRT optimizes the model graph and applies hardware-specific optimizations, typically providing 5-10x speedup over CPU inference.

INT8 quantization would further reduce model size and improve inference speed with minimal accuracy loss. On the tracking side, the SAHI slice size can be increased from 320x320 to 512x512 to reduce the number of tiles per frame (from 15 to approximately 6), significantly improving throughput while accepting a minor reduction in small-object detection accuracy.

6. Limitations

• CPU inference at 4.83 FPS is insufficient for real-time deployment — GPU acceleration required

• SAHI reduces FPS to 0.24 on CPU — practical deployment requires GPU or TensorRT optimization

• Model was not fine-tuned on VisDrone training data — fine-tuning would improve accuracy significantly

• Night and low-light performance was not tested

• ID switching still occurs on fast-moving or heavily occluded persons

• SAHI introduces duplicate detections at tile boundaries — NMS mitigates but does not fully eliminate this

7. What I Would Do With More Time

• Fine-tune YOLOv8n on the VisDrone training set for significantly better small-person detection

• Test YOLOv8s (small) which offers better accuracy and is still within the 300MB model size limit

• Add a re-identification module for long-term tracking across occlusions

• Implement INT8 quantization for faster CPU inference

• Evaluate on all 7 sequences and compute MOTA/MOTP metrics

• Experiment with larger SAHI overlap ratios to reduce missed detections at tile boundaries

8. References

[1] Zhang, Y. et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. ECCV 2022.

[2] Jocher, G. et al. Ultralytics YOLOv8. GitHub, 2023. https://github.com/ultralytics/ultralytics

[3] Akyon, F.C. et al. Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection. ICIP 2022.

[4] Zhu, P. et al. VisDrone-DET2019: The Vision Meets Drone Object Detection in Image Challenge Results. ICCV Workshop 2019.

[5] Bewley, A. et al. Simple Online and Realtime Tracking. ICIP 2016.